

Project Reflection

One important design decision was to separate different types of external capabilities according to their purpose. I used the AgentCore Gateway with MCPClient for customer and order operations because these tools represent backend business systems that the support agent needs to access dynamically. For product and policy questions, I implemented a dedicated `search_knowledge_base` tool that calls the Bedrock Knowledge Base Retrieve API. This keeps informational responses grounded in the product catalog instead of relying only on the language model's internal knowledge. I also used the AgentCore Code Interpreter for loyalty calculations so that points redemption, tier discounts, and order limits are calculated deterministically.

One challenge I encountered was deployment compatibility. The original project workflow used the Python-based AgentCore starter toolkit, but the environment was affected by an explicit IAM deny associated with the laboratory permissions. Deployment attempts reached the CodeBuild stage but failed because required IAM and CodeBuild actions were explicitly denied. I also encountered a browser navigation reliability issue during local testing and adjusted the agent so browser requests are routed to a dedicated browser-oriented agent rather than unnecessarily combining browser operations with the Gateway tools.

A major production consideration is security and cost. The educational Gateway uses a NONE authorizer, which would not be appropriate for a real customer-support application. In production, I would use authenticated access, least-privilege IAM policies, input validation, structured tool-response validation, monitoring, and audit logging. I would also control model and retrieval costs through appropriate session management, caching, and conversation summarization. These measures would make the architecture more secure, scalable, observable, and suitable for real customer-facing workloads.